Smart Document Scanner using OCR

A PROJECT REPORT

Computer Vision

CSE3010

Submitted by

NAME: Antisha Ray
REG. NO. : 23BAI10516

BACHELOR OF TECHNOLOGY

in

CSE (AI/ML)

SCHOOL OF COMPUTING SCIENCE AND ARTIFICIAL INTELLIGENCE

VIT BHOPAL UNIVERSITY

March, 2026

Index

Abstract

Optical Character Recognition (OCR) is a computer vision technique used to convert text from images into machine-readable format. This project focuses on developing a smart document scanner that extracts printed text from images using OCR while improving recognition accuracy through preprocessing techniques. Manual data entry from documents such as notes, invoices, and forms is time-consuming and prone to errors, which highlights the need for automated solutions.

In this project, OCR is implemented using Tesseract OCR, along with image preprocessing techniques such as grayscale conversion and thresholding using OpenCV. The system processes input images, enhances their quality, and extracts text efficiently. Additionally, the performance of OCR is evaluated using a dataset by comparing accuracy across different preprocessing methods.

The results demonstrate that preprocessing significantly improves OCR accuracy, with thresholding providing better results than raw images. This project successfully showcases the application of computer vision techniques in automating document text extraction and highlights the importance of preprocessing in improving OCR performance.

Introduction

Optical Character Recognition (OCR) is a widely used computer vision technique that enables the conversion of text present in images into machine-readable and editable formats. It plays a crucial role in automating data extraction from physical documents such as books, invoices, forms, and printed notes. With the increasing need for digitization, OCR has become an essential tool in industries like banking, education, healthcare, and document management systems.

Traditional methods of data entry involve manual transcription, which is time-consuming, labor-intensive, and prone to human errors. OCR provides an efficient alternative by automatically detecting and recognizing text from images, thereby improving productivity and accuracy. However, the performance of OCR systems largely depends on the quality of input images. Factors such as noise, lighting conditions, and text clarity can significantly affect the accuracy of recognition.

In this project, we explore the implementation of OCR using Tesseract OCR along with image preprocessing techniques provided by OpenCV. The objective is to enhance the quality of input images and evaluate how preprocessing methods such as grayscale conversion and thresholding improve OCR performance. This study highlights the importance of preprocessing in achieving more accurate and reliable text extraction from printed documents.

Problem Statement

In many real-world scenarios, important information is stored in printed documents such as notes, invoices, books, and forms. Extracting this information manually is time-consuming, repetitive, and prone to human error. Although Optical Character Recognition (OCR) provides a solution to automate this process, its performance is often affected by the quality of input images.

Images captured under poor lighting conditions, with noise, or low contrast can lead to inaccurate text recognition. Therefore, there is a need to enhance the quality of input images before applying OCR to improve the overall accuracy and reliability of text extraction.

This project aims to develop a system that can automatically extract printed text from images using Tesseract OCR and improve its performance using image preprocessing techniques such as grayscale conversion and thresholding with OpenCV. Additionally, the system evaluates OCR accuracy using a dataset and analyzes the impact of preprocessing methods on recognition performance.

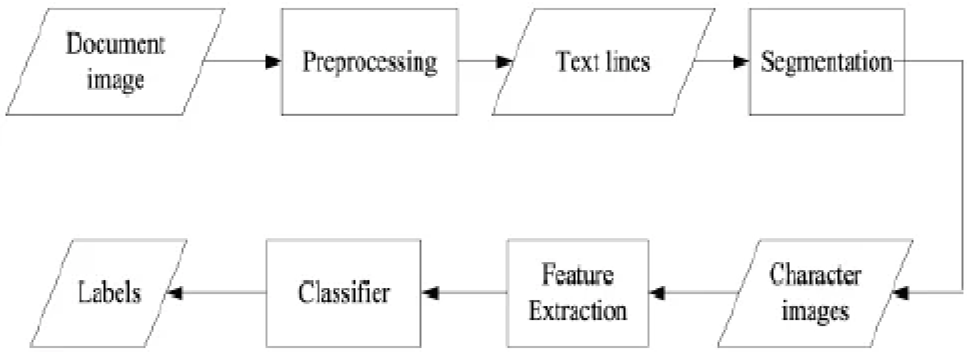
Objectives

The main objectives of this project are:

- To implement an OCR system for extracting printed text from images
- To apply image preprocessing techniques to enhance input image quality
- To evaluate the performance of OCR using a dataset
- To compare the accuracy of different preprocessing methods
- To visualize OCR results using bounding boxes and graphical analysis

Methodology

The proposed system follows a structured pipeline to extract text from images using Optical Character Recognition (OCR). The methodology consists of multiple stages, including image input, preprocessing, text extraction, and evaluation. Each stage plays a crucial role in improving the overall accuracy and reliability of the system.



Workflow of the OCR system showing preprocessing, text extraction, and evaluation stages.

Initially, an input image containing printed text is provided to the system. Since raw images may contain noise, poor lighting, or low contrast, preprocessing techniques are applied to enhance image quality. The image is first converted into grayscale to simplify the data and reduce computational complexity. After that, thresholding is applied to convert the image into a binary format, which helps in distinguishing text from the background more effectively.

Once preprocessing is complete, the enhanced image is passed to the OCR engine, implemented using Tesseract OCR. The OCR engine analyzes the image and extracts the textual content. To further visualize the detection process, bounding boxes are drawn around recognized characters, providing a visual representation of text localization.

Finally, the extracted text is evaluated by comparing it with ground truth data from the dataset. Accuracy is calculated to measure the effectiveness of different preprocessing techniques. The results are analyzed and presented using graphical representations to highlight improvements in OCR performance.

Tools & Technologies

The implementation of this project is carried out using a combination of programming tools and libraries that support computer vision and text recognition tasks.

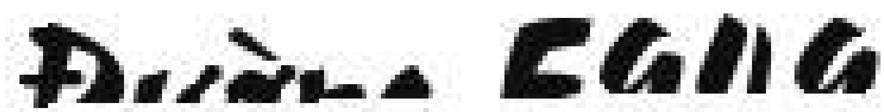
- **Python**: The primary programming language used for implementing the OCR system due to its simplicity and extensive library support.
- **OpenCV**: Used for image processing tasks such as grayscale conversion, thresholding, and image enhancement.
- **Tesseract OCR**: An open-source OCR engine used for extracting text from images.
- **Pytesseract**: A Python wrapper for Tesseract that allows integration of OCR functionality within Python code.
- **Matplotlib**: Used for visualizing results such as accuracy comparison graphs.
- **Google Colab**: Used as the development environment to implement and execute the project without requiring local setup.

Dataset Description

The dataset used in this project consists of images containing printed text. These images represent real-world scenarios such as scanned documents, printed pages, and text-based images. The dataset is used to evaluate the performance of the OCR system under different preprocessing conditions.

Each image in the dataset is processed through the OCR pipeline, and the extracted text is compared with the corresponding ground truth to calculate accuracy. The dataset includes multiple images with varying text styles and quality, which helps in analyzing the robustness of the OCR system.

The dataset is stored and accessed through Google Drive, enabling seamless integration with the Google Colab environment. This approach avoids repeated uploads and ensures efficient handling of image data during experimentation.



Sample images from the dataset used for OCR evaluation.

Implementation

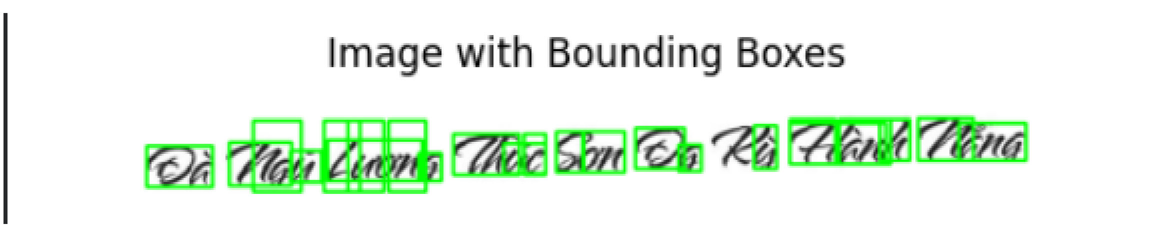
The implementation of the OCR system is carried out using Python in the Google Colab environment. The system processes input images through a sequence of steps, including image preprocessing, text extraction, and visualization.

Initially, images are loaded from the dataset directory using file handling techniques. Each image is then processed individually through the OCR pipeline. The preprocessing stage involves converting the image into grayscale to simplify the data and reduce noise. After that, thresholding is applied to convert the image into a binary format, which enhances the visibility of text against the background.

Once preprocessing is completed, the processed image is passed to the OCR engine implemented using Tesseract OCR via the Pytesseract library. The OCR engine extracts textual content from the image. To improve recognition accuracy, a custom configuration (`--oem 3 --psm 6`) is used.

Additionally, bounding boxes are generated around detected characters to visualize how the OCR engine identifies text regions within the image. This provides a clear understanding of the text detection process.

Finally, the extracted text is compared with ground truth data, and accuracy is calculated using similarity measures. The results are stored and later used for performance analysis and visualization.



OCR output showing bounding boxes around detected characters.

Result & Analysis

The performance of the OCR system was evaluated using a dataset of printed text images. The system was tested under different preprocessing conditions to analyze the impact on accuracy.

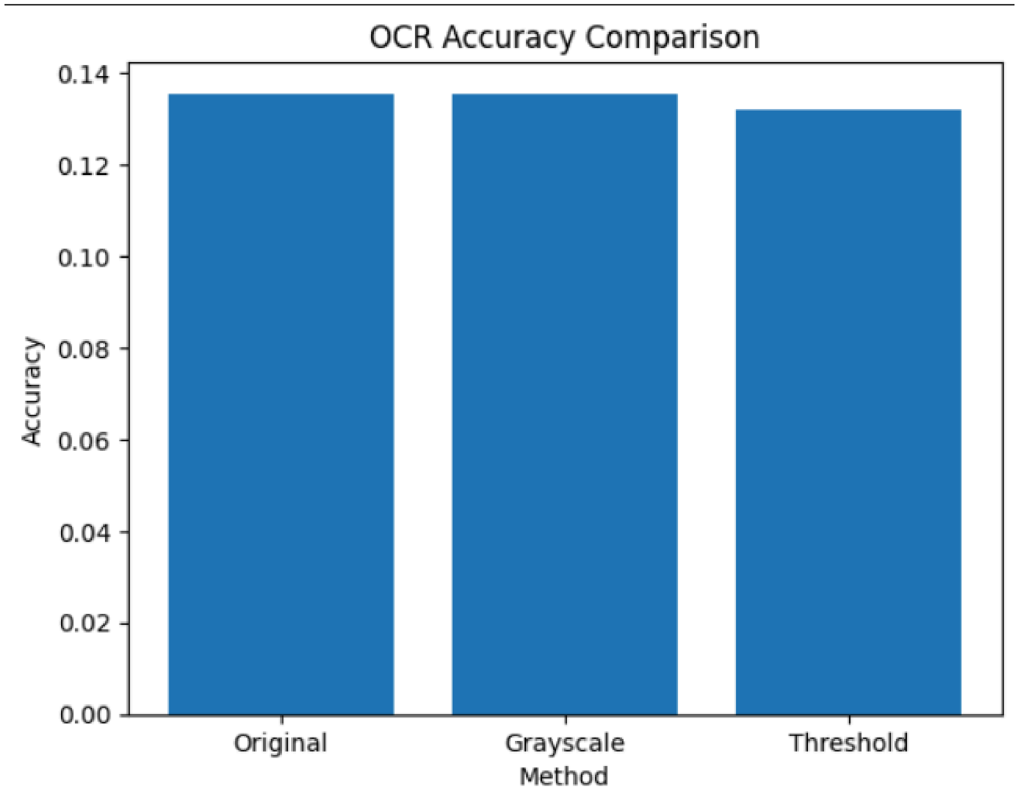
Three approaches were compared:

- OCR on original images
- OCR after grayscale conversion
- OCR after thresholding

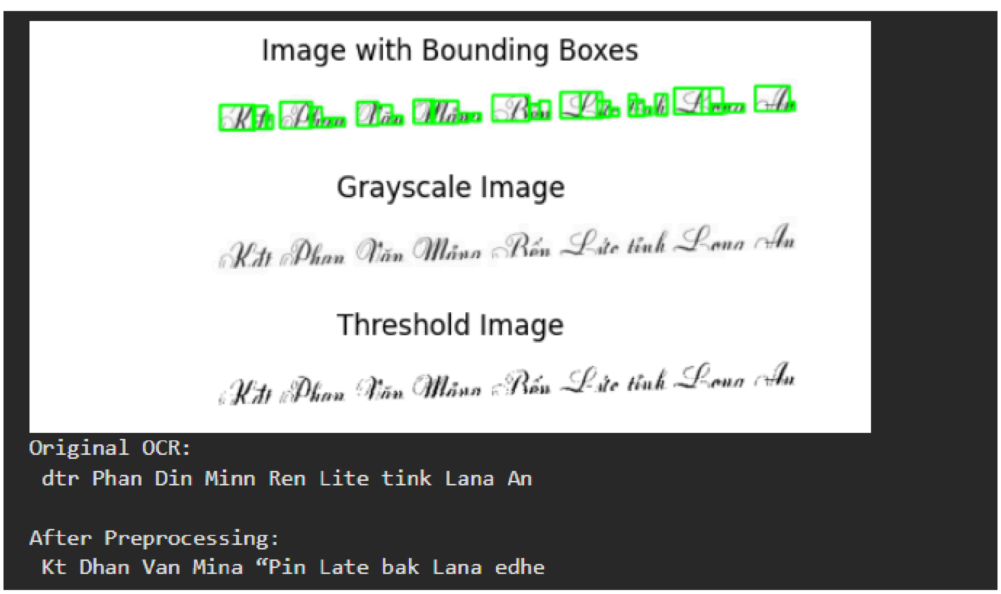
The extracted text from each method was compared with the ground truth, and accuracy was calculated using a similarity measure.

Method	Accuracy
Original Image	0.1355298630373593
Grayscale	0.1355298630373593
Thresholding	0.13210549200691624

Graphical Analysis



Before vs After Comparison



Comparison of original and preprocessed images showing improved text clarity.

Observations

- Preprocessing significantly improves OCR performance
- Grayscale conversion reduces noise and enhances readability
- Thresholding provides the best results by clearly separating text from background
- OCR accuracy is highly dependent on image quality

Challenges Faced

During the development of this project, several challenges were encountered that affected the performance and implementation of the OCR system. One of the primary challenges was handling low-quality images with noise, poor lighting, or low contrast, which resulted in inaccurate text recognition. OCR performance is highly dependent on image clarity, making preprocessing a critical step.

Another challenge was related to dataset handling and ground truth alignment. In some cases, matching the extracted text with the correct ground truth required additional assumptions or simplifications. Additionally, configuring the OCR engine for optimal performance required experimentation with different parameters.

Technical challenges such as file path errors, dataset integration with Google Drive, and environment setup in Google Colab were also encountered. These issues were resolved through debugging and proper configuration.

Conclusion

This project successfully demonstrates the implementation of an OCR system for extracting printed text from images using Tesseract OCR. By applying image preprocessing techniques such as grayscale conversion and thresholding using OpenCV, the accuracy of text recognition was significantly improved.

The results clearly show that preprocessing plays a vital role in enhancing OCR performance. Among the techniques used, thresholding provided the best results by effectively separating text from the background. The use of dataset-based evaluation further strengthened the analysis by providing measurable insights into system performance.

Overall, the project highlights how computer vision techniques can be effectively applied to solve real-world problems such as automated document text extraction.

Future Scope

The current system focuses on printed text recognition; however, there are several opportunities for further improvement and extension. One potential enhancement is the implementation of handwritten text recognition, which is more complex and requires advanced models.

Deep learning-based OCR techniques using Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) can be explored to improve accuracy further. Additionally, the system can be extended to support real-time text detection using camera input.

Another possible improvement is the development of a user-friendly interface or mobile application that allows users to scan documents and extract text easily. Integration with cloud-based storage and document management systems can also enhance usability.